

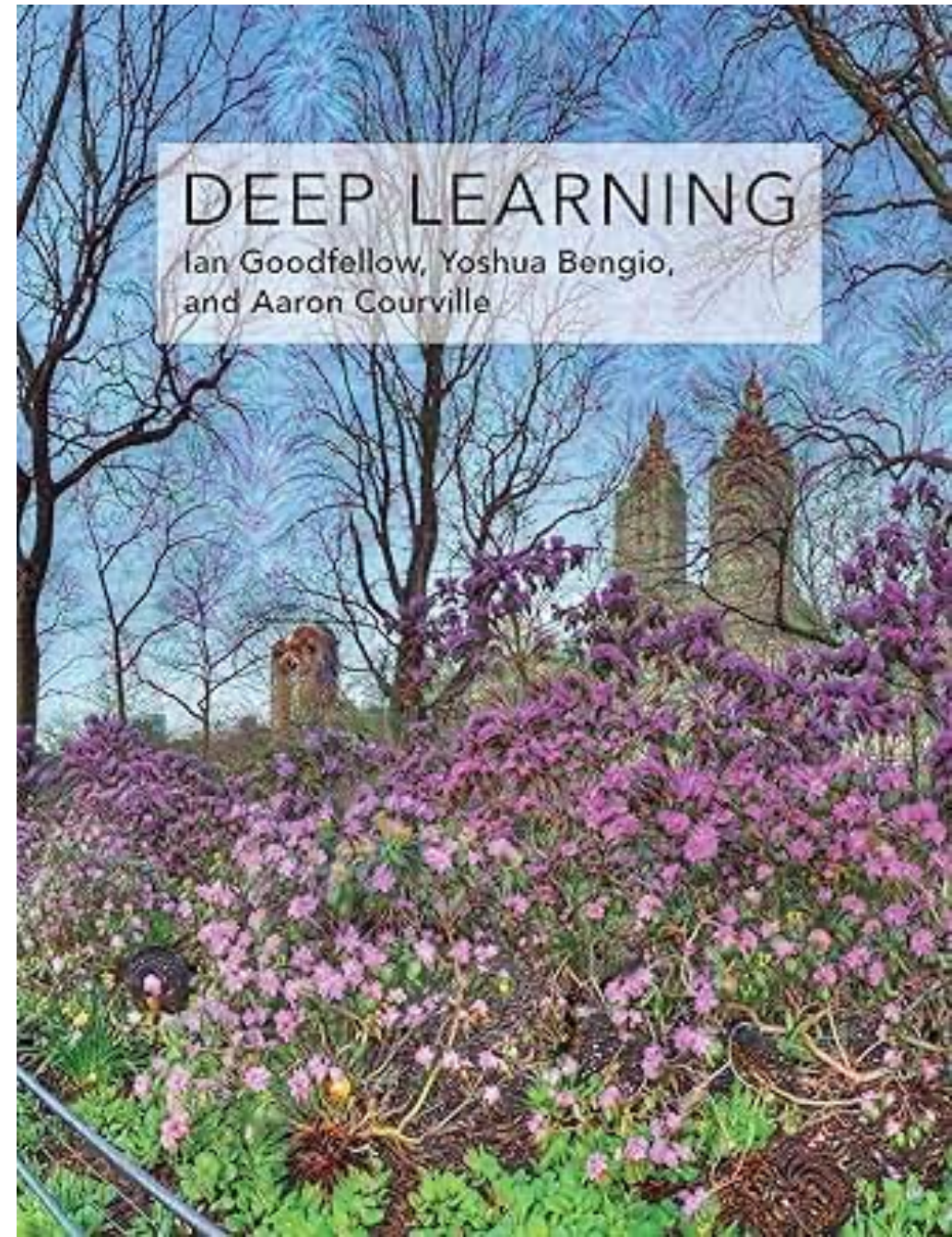


CSCI 6032: Lecture 4: Agents

Instructor: Dr. David A. G. Harrison

Things I want to cover today...

- **Agents**
 - Setting up GitHub Copilot CLI
 - What makes a CLI agent different from a chat window
 - Tools, permissions, and the agent loop
 - Skills: packaging repeatable workflows
- **Do it securely.**



Office Hours

207 Weir Hall

Monday	11:00 AM-12:00 PM
Wednesday	1:00-2:00 PM

A vertical banner on the left side of the slide with the word "HOLLYWOOD" written vertically in a stylized font. The letters are white and blue, set against a red background.

HW1 (Reminder)

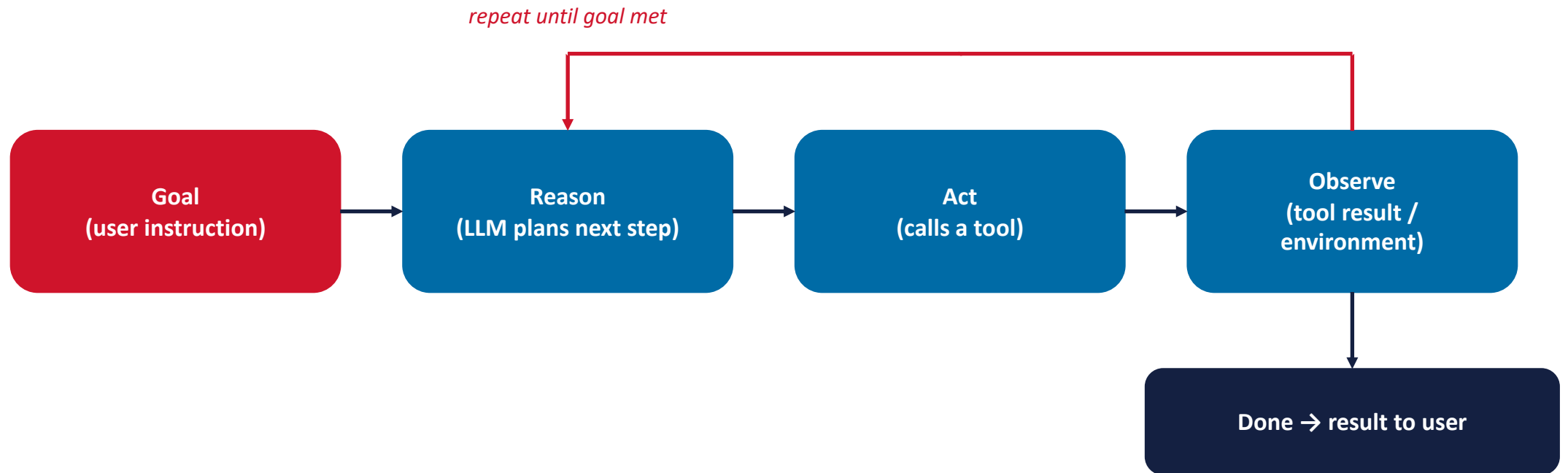
On github already...

Due today at 11pm

Submit as notebook via blackboard.

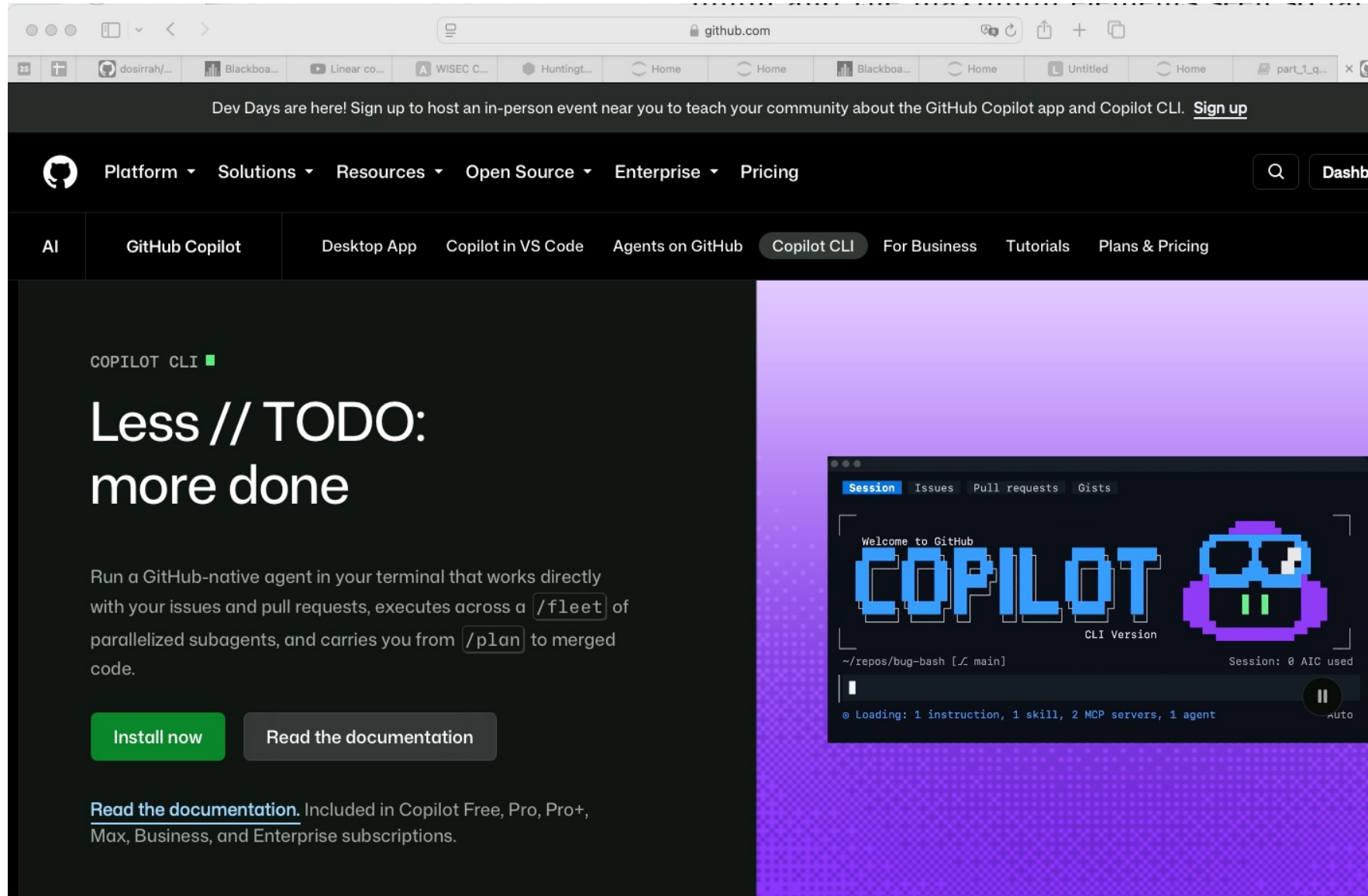
What Is an Agent?

A program that loops: reason about the goal, act in the world, observe what happened — and repeat until the goal is done.



Agent = model (reasoning) + tools (ability to act) + a loop, operating with some autonomy toward a goal.

Agents: GitHub Copilot



Dev Days are here! Sign up to host an in-person event near you to teach your community about the GitHub Copilot app and Copilot CLI. [Sign up](#)

Platform Solutions Resources Open Source Enterprise Pricing

AI GitHub Copilot Desktop App Copilot in VS Code Agents on GitHub Copilot CLI For Business Tutorials Plans & Pricing

COPILOT CLI

Less // TODO: more done

Run a GitHub-native agent in your terminal that works directly with your issues and pull requests, executes across a `/fleet` of parallelized subagents, and carries you from `/plan` to merged code.

[Install now](#) [Read the documentation](#)

[Read the documentation.](#) Included in Copilot Free, Pro, Pro+, Max, Business, and Enterprise subscriptions.



Ole Miss

Agents: GitHub Copilot

<https://github.com/settings/education/benefits>



David Harrison (dosirrah)

Your personal account

 Menu

 Switch context ▾

**Free Student
edition.**

GitHub Education

Free GitHub developer resources for students and teachers

Get Copilot for free, 180 monthly Codespaces hours for cloud coding, unlimited private repositories with GitHub Pro or Team, and dozens of premium tools in the Student Developer Pack.

GitHub Copilot

Command-Line Interface

- **Runs in terminal (shell)**
- **Can install in linux, windows, or macOS.**

Use one of these commands:

- **Cross-platform (npm)**

Prerequisite: Node.js 22 or later.

Bash

```
npm install -g @github/copilot
```

- **Windows (WinGet)**

Bash

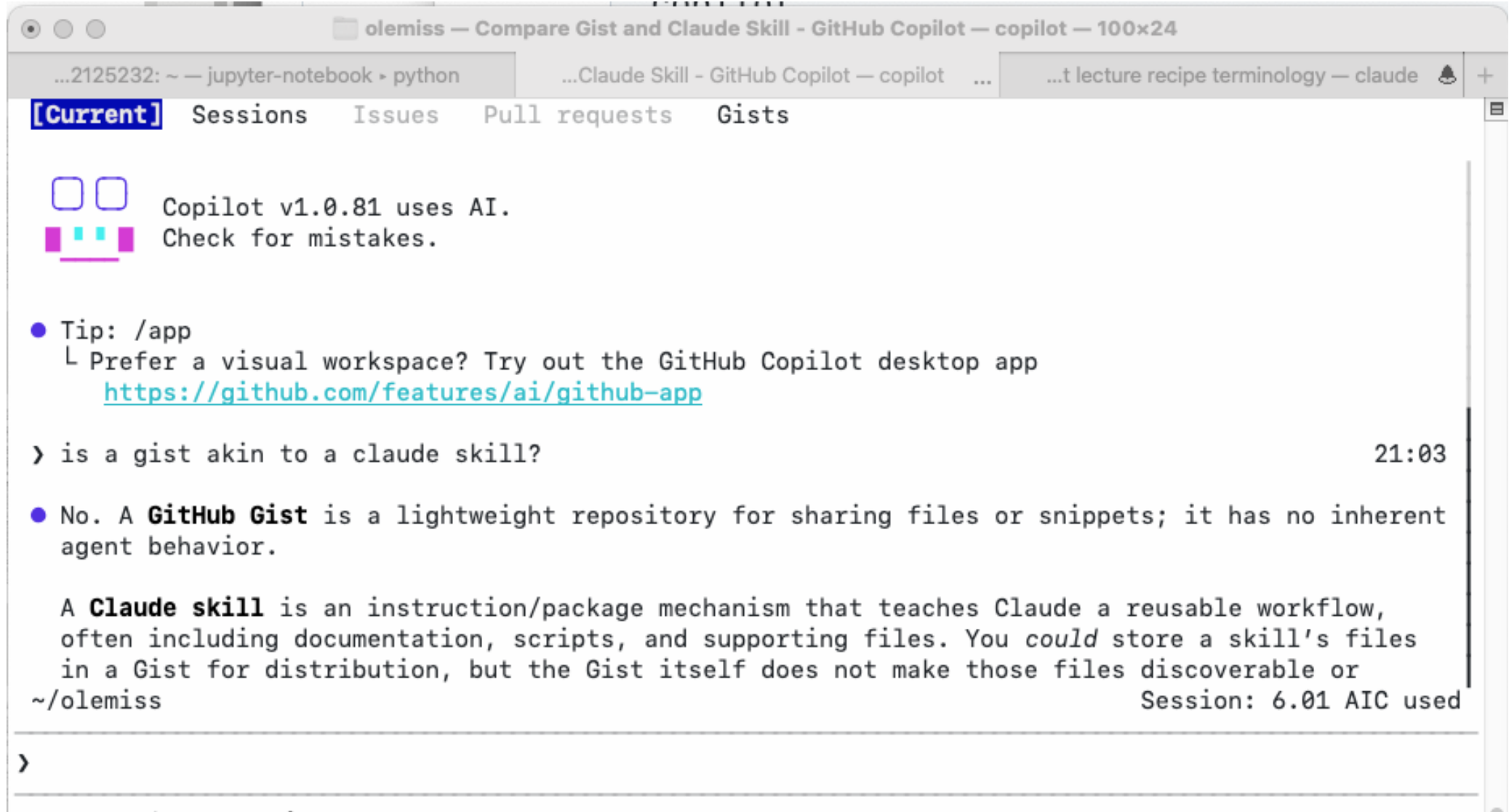
```
winget install GitHub.Copilot
```

- **macOS/Linux (Homebrew)**

Bash

```
brew install --cask copilot-cli
```


GitHub Copilot



```
olemiss — Compare Gist and Claude Skill - GitHub Copilot — copilot — 100x24
...2125232: ~ — jupyter-notebook > python
...Claude Skill - GitHub Copilot — copilot
...t lecture recipe terminology — claude
[Current] Sessions Issues Pull requests Gists

Copilot v1.0.81 uses AI.
Check for mistakes.

• Tip: /app
  ↳ Prefer a visual workspace? Try out the GitHub Copilot desktop app
  https://github.com/features/ai/github-app

> is a gist akin to a claude skill? 21:03

• No. A GitHub Gist is a lightweight repository for sharing files or snippets; it has no inherent agent behavior.

A Claude skill is an instruction/package mechanism that teaches Claude a reusable workflow, often including documentation, scripts, and supporting files. You could store a skill's files in a Gist for distribution, but the Gist itself does not make those files discoverable or
~/olemiss Session: 6.01 AIC used

>
```

What Makes a CLI Agent Different from a Chat Window?

A chat window answers questions; a CLI agent takes action

- Chat: you copy code out, run it, and paste results back in
- CLI agent: reads, edits, and runs code directly in your project

It can take multiple steps on its own

- Plans, calls tools, checks results, and continues — without a relay

It needs permission boundaries

- Because it acts instead of just talking, what it's allowed to do matters

Tools

A tool is a specific capability the agent can invoke

- Read / write / edit files
- Run shell commands
- Search the web, browse a page
- Query an API, a database, a service

The agent decides which tool to call and with what arguments

Tool results feed back into its reasoning for the next step

More tools = more capability, but also more surface area for mistakes

Permissions

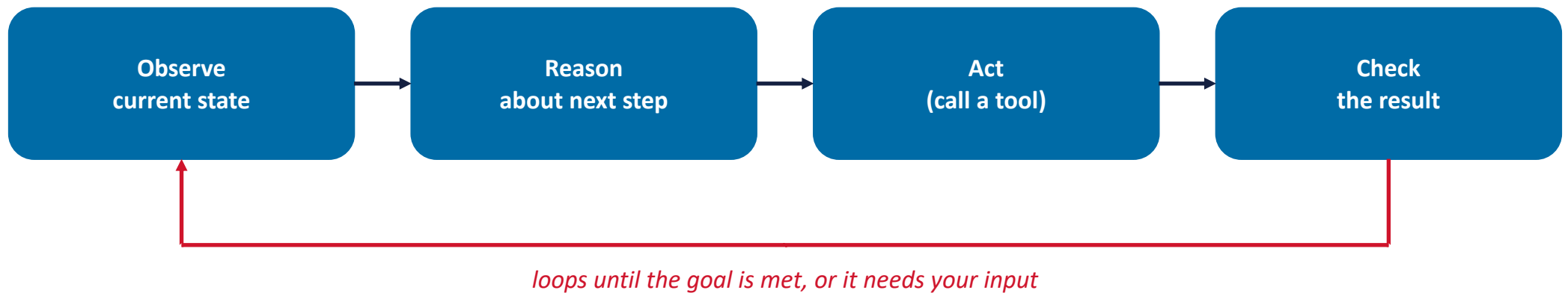
Every tool call is a real action — permissions decide which ones need a human first

- **Auto-approved**
 - Safe, reversible actions (e.g., reading a file, listing a directory)
- **Ask first**
 - Risky or hard-to-reverse actions (e.g., editing files, running a command, pushing to git)
- **Denied**
 - Actions explicitly out of bounds for the task

Good permission design assumes the agent will eventually be wrong about something

- You're bounding the blast radius of a mistake, not just approving good behavior

The Agent Loop



- Each iteration, the agent decides: keep going, ask the user, or stop.
- It stops when the goal is met, it hits a permission boundary, or it's stuck and needs help.
- This is what separates an agent from a single autocomplete suggestion.

Skills: Packaging Repeatable Workflows

A skill is a saved, reusable recipe for a task you do more than once

- Instructions the agent follows step by step (e.g., “post a lecture”, “grade a homework”)
- Can bundle scripts, templates, and checklists alongside the instructions

Why bother?

- Consistency — the same steps happen the same way every time
- Less re-explaining — you don't retype the process in every session
- Shareable — teammates (or future you) can reuse the same workflow

Creating a Skill

A skill is just a folder with instructions the agent can discover and load

- A short description of when to use it (its trigger)
- Step-by-step instructions written in plain language
- Optional helper scripts or reference files it can call

The agent decides on its own when a skill applies

- You describe the task; if it matches a skill's description, the agent invokes it

Good skills are narrow and explicit

- One clear workflow per skill beats one skill that tries to do everything

Why USE PYTHON FOR Machine learning?

Python is easy to program.

Python has beautiful, terse syntax.

Python is well supported by tools.

The big stuff is usually not done in Python, so slow doesn't matter (usually):

- **PyTorch**
- **TensorFlow**

The medium-sized stuff and postprocessing is usually not done in Python either

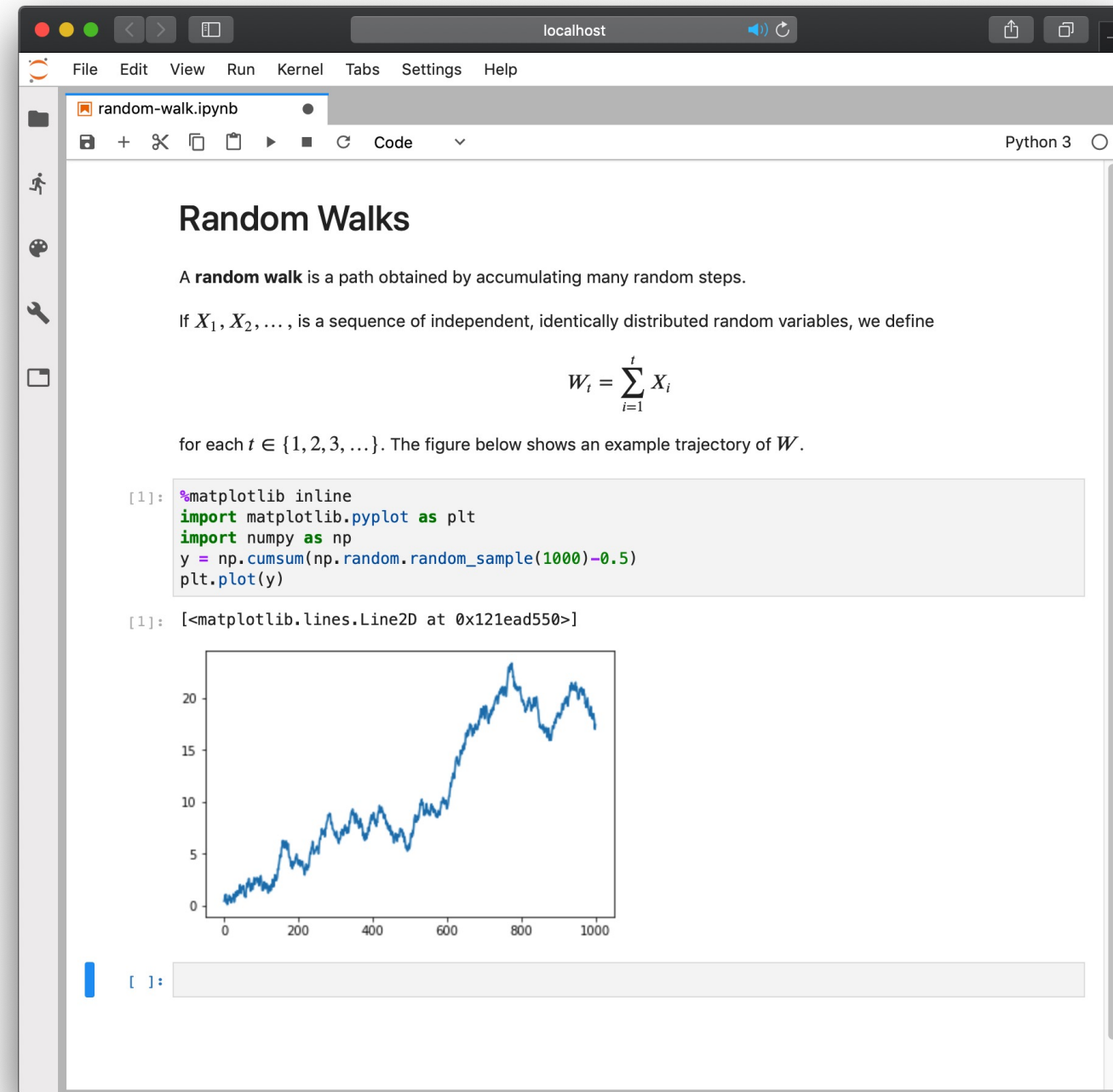
- **NumPy**
- **DataFrames**
- **SciPy**

Why use jupyter?

All of the benefits of Python.

The ability to mix visualization, description (markdown), math (latex), and code in the same environment: a notebook.

Ability to run the front end locally or remotely.

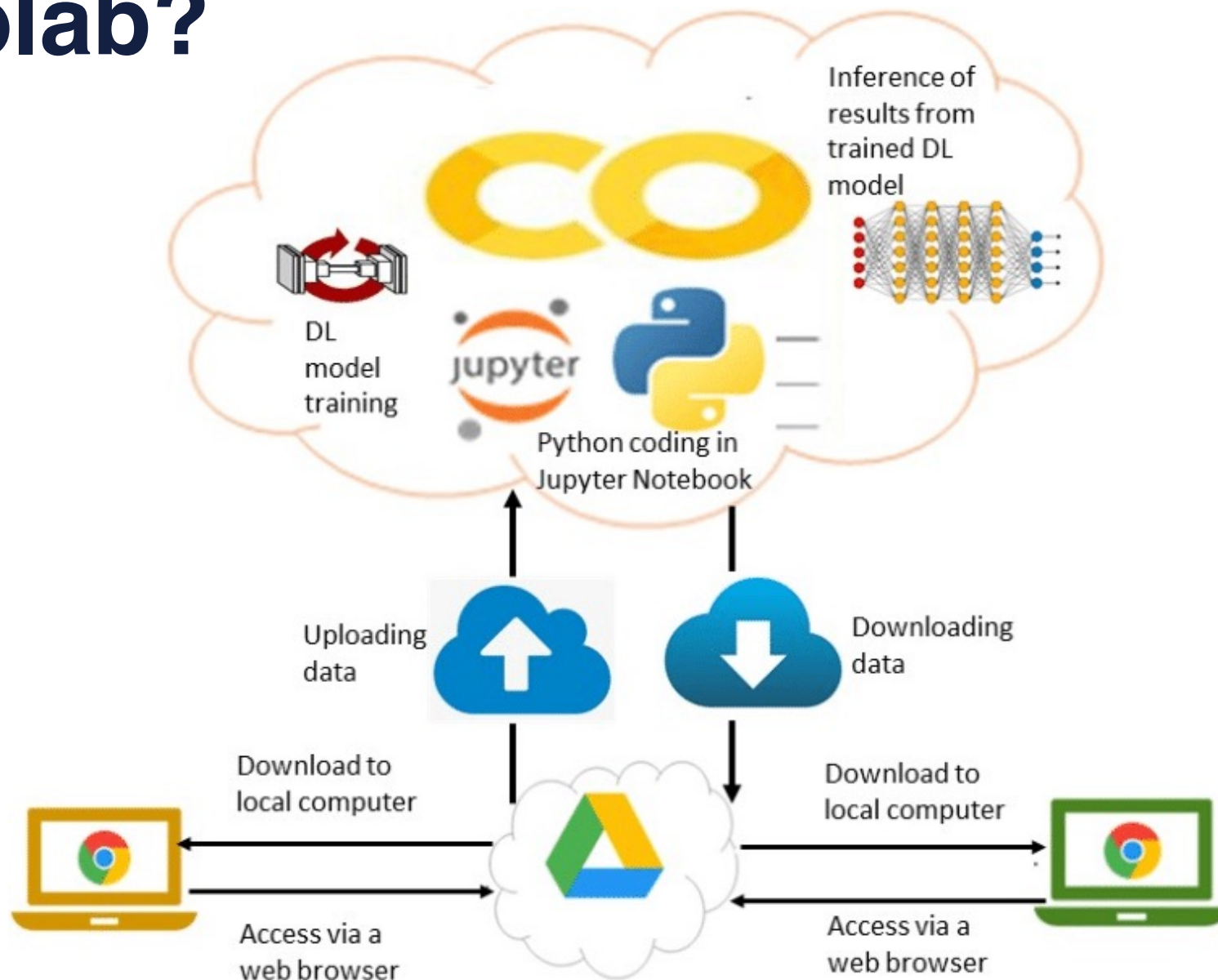


Why use Colab?

All of the benefits of jupyter and python but run remotely.

Can collaborate by sharing the notebook.

Ability to train and validate machine learning algorithms across a cluster (e.g., using PyTorch or TensorFlow) while displaying the training and validation in a notebook.

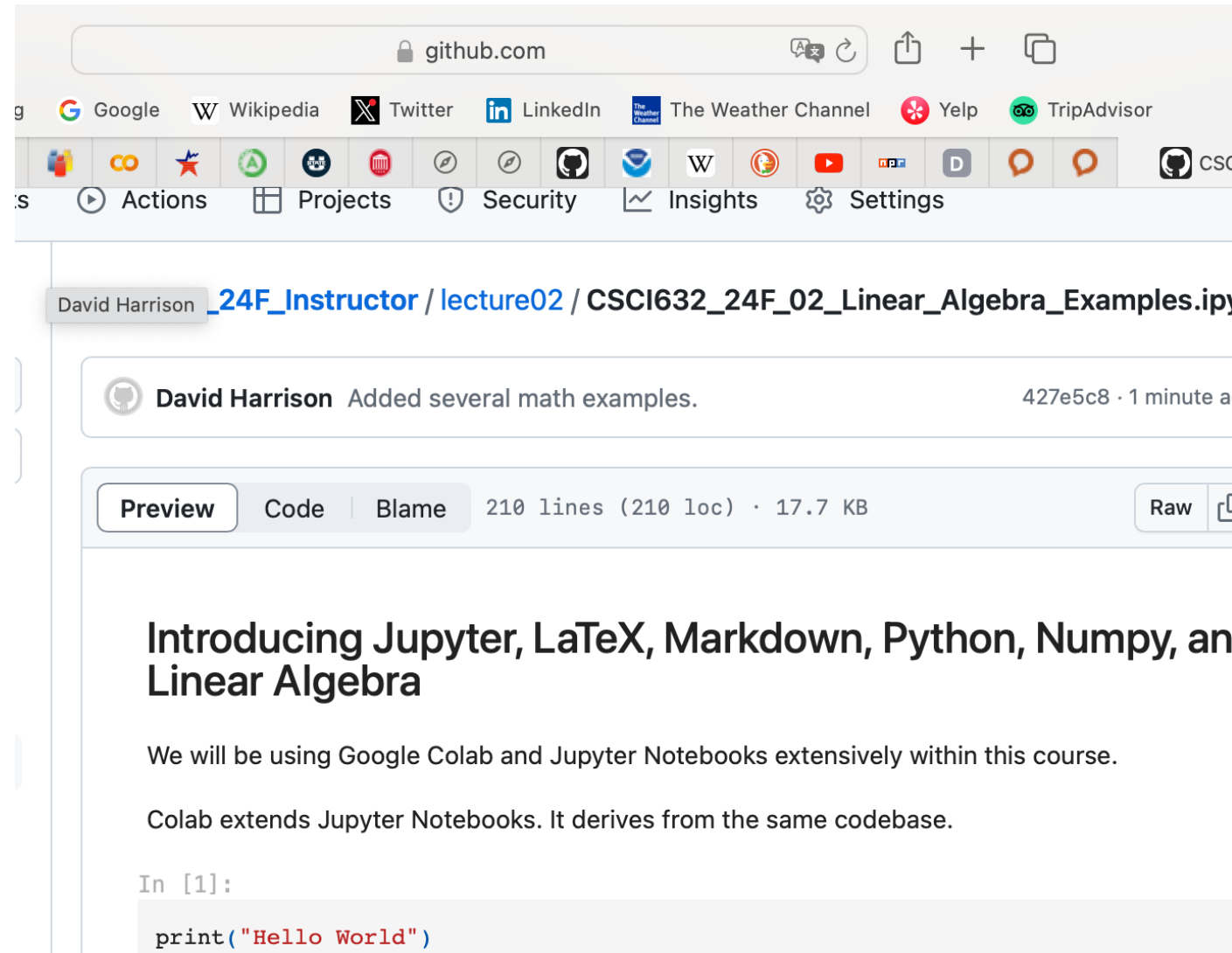


Jupyter and LaTeX

See Jupyter notebook
Similar can be done
in Google Colab

You can open this
notebook in GitHub
WITHOUT installing
jupyter on your local
system.

https://github.com/dosirrah/CSCI6032_26F_Instructor/blob/main/lecture02/CSCI6032_26F_02_Linear_Algebra_Examples.ipynb



The screenshot shows a web browser displaying the GitHub interface. The address bar shows 'github.com'. The page header includes navigation links like 'Actions', 'Projects', 'Security', 'Insights', and 'Settings'. The main content area shows a commit by 'David Harrison' titled '_24F_Instructor / lecture02 / CSCI632_24F_02_Linear_Algebra_Examples.ipynb'. Below the commit, there are tabs for 'Preview', 'Code', and 'Blame'. The 'Preview' tab is selected, showing the content of the Jupyter notebook. The notebook title is 'Introducing Jupyter, LaTeX, Markdown, Python, Numpy, and Linear Algebra'. The text in the notebook says: 'We will be using Google Colab and Jupyter Notebooks extensively within this course. Colab extends Jupyter Notebooks. It derives from the same codebase.' Below this, there is a code cell with the following content: 'In [1]: print("Hello World")'.

github.com

Google Wikipedia Twitter LinkedIn The Weather Channel Yelp TripAdvisor

Actions Projects Security Insights Settings

David Harrison _24F_Instructor / lecture02 / CSCI632_24F_02_Linear_Algebra_Examples.ipynb

David Harrison Added several math examples. 427e5c8 · 1 minute ago

Preview Code Blame 210 lines (210 loc) · 17.7 KB Raw

Introducing Jupyter, LaTeX, Markdown, Python, Numpy, and Linear Algebra

We will be using Google Colab and Jupyter Notebooks extensively within this course.

Colab extends Jupyter Notebooks. It derives from the same codebase.

In [1]:

```
print("Hello World")
```

Notation

We will use the notation from Goodfellow et al, *Deep Learning*.

To the right is just the first page of notation from the book.

<https://www.deeplearningbook.org/contents/notation.html>

Numbers and Arrays

a	A scalar (integer or real)
$\mathbf{a}$	A vector
$\mathbf{A}$	A matrix
$\mathbf{A}$	A tensor
$\mathbf{I}_n$	Identity matrix with n rows and n columns
$\mathbf{I}$	Identity matrix with dimensionality implied by context
$\mathbf{e}^{(i)}$	Standard basis vector $[0, \dots, 0, 1, 0, \dots, 0]$ with a 1 at position i
$\text{diag}(\mathbf{a})$	A square, diagonal matrix with diagonal entries given by $\mathbf{a}$
a	A scalar random variable
$\mathbf{a}$	A vector-valued random variable
$\mathbf{A}$	A matrix-valued random variable